

Checkpoint 3

1. SQL Queries / SQL Query Level Requirement

Query 1: Player Search with Multiple Filters (Level 3)

sql

```
SELECT
    p.player_id,
    p.name AS player_name,
    c.name AS club_name,
    co.name AS nationality,
    p.position,
    p.sub_position,
    AVG(a.goals) AS avg_goals_per_game,
    AVG(a.assists) AS avg_assists_per_game,
    AVG(a.minutes_played) AS avg_minutes_played,
    COUNT(DISTINCT a.game_id) AS games_played
FROM Player p
LEFT JOIN Club c ON p.current_club_id = c.club_id
LEFT JOIN Country co ON p.country_id = co.country_id
LEFT JOIN Appearance a ON p.player_id = a.player_id
WHERE
    (p.position = 'Attacker' OR p.position = 'Midfielder')
    AND p.date_of_birth >= '1995-01-01'
GROUP BY p.player_id, p.name, c.name, co.name, p.position, p.sub_position
HAVING AVG(a.goals) > 0.5
ORDER BY AVG(a.goals) DESC, games_played DESC
LIMIT 20;
```

Description: This query finds attacking players (forwards and midfielders) born after 1995 who average more than 0.5 goals per game. It returns their basic info, club, nationality, and

aggregated performance metrics. Scouts and fantasy football players can identify promising young attacking talent with consistent goal-scoring records across multiple games. The filters allow users to narrow down by position and age range.

Query Level Justification: Level 3 - Uses multiple joins (Player, Club, Country, Appearance), aggregate functions (AVG, COUNT), GROUP BY, HAVING with aggregate condition, and complex WHERE filtering.

Query 2: Player Market Value Trend Analysis (Level 3)

sql

```
SELECT
    p.name AS player_name,
    c.name AS club_name,
    pv.date AS valuation_date,
    pv.market_value_eur,
    LAG(pv.market_value_eur) OVER (PARTITION BY pv.player_id ORDER BY pv.date) AS
previous_value,
    ROUND((((pv.market_value_eur - LAG(pv.market_value_eur) OVER (PARTITION BY
pv.player_id ORDER BY pv.date))) /
        LAG(pv.market_value_eur) OVER (PARTITION BY pv.player_id ORDER BY pv.date)))
* 100, 2) AS value_change_pct,
    DATEDIFF(pv.date, LAG(pv.date) OVER (PARTITION BY pv.player_id ORDER BY
pv.date)) AS days_between
FROM PlayerValuation pv
JOIN Player p ON pv.player_id = p.player_id
JOIN Club c ON pv.current_club_id = c.club_id
WHERE p.name LIKE '%Mbappé%'
ORDER BY pv.date DESC;
```

Description: Shows the market value history for a specific player with percentage changes between valuations and the time between updates. Scouts and analysts can see how a player's

market value has evolved over time, identify value spikes or drops, and correlate value changes with transfers or performance.

Query Level Justification: Level 3 - Uses window function (LAG with PARTITION BY), complex calculations (percentage change), and multiple joins.

Query 3: Head-to-Head Player Comparison (Level 3)

sql

```
WITH player1_stats AS (  
    SELECT  
        p.player_id,  
        p.name,  
        COUNT(DISTINCT a.game_id) AS games_played,  
        SUM(a.goals) AS total_goals,  
        SUM(a.assists) AS total_assists,  
        SUM(a.minutes_played) AS total_minutes,  
        AVG(a.goals) AS avg_goals,  
        AVG(a.assists) AS avg_assists,  
        AVG(a.minutes_played) AS avg_minutes  
    FROM Player p  
    JOIN Appearance a ON p.player_id = a.player_id  
    WHERE p.player_id = 12345 -- First player ID  
    GROUP BY p.player_id, p.name  
,  
player2_stats AS (  
    SELECT  
        p.player_id,  
        p.name,  
        COUNT(DISTINCT a.game_id) AS games_played,  
        SUM(a.goals) AS total_goals,  
        SUM(a.assists) AS total_assists,  
        SUM(a.minutes_played) AS total_minutes,
```

```

        AVG(a.goals) AS avg_goals,
        AVG(a.assists) AS avg_assists,
        AVG(a.minutes_played) AS avg_minutes
FROM Player p
JOIN Appearance a ON p.player_id = a.player_id
WHERE p.player_id = 23456 -- Second player ID
GROUP BY p.player_id, p.name
)
SELECT
    'Player 1' AS comparison_side,
    p1.name AS player_name,
    p1.games_played,
    p1.total_goals,
    p1.total_assists,
    p1.avg_goals,
    p1.avg_assists,
    p1.total_minutes
FROM player1_stats p1
UNION ALL
SELECT
    'Player 2' AS comparison_side,
    p2.name AS player_name,
    p2.games_played,
    p2.total_goals,
    p2.total_assists,
    p2.avg_goals,
    p2.avg_assists,
    p2.total_minutes
FROM player2_stats p2;

```

Description: Compares two players side-by-side using a common table expression (CTE) for each player, showing games played, total goals/assists, and averages. Users can directly compare players to make informed decisions for fantasy football picks, scouting, or transfer decisions.

Query Level Justification: Level 3 - Uses CTEs (subqueries in WITH clause), multiple joins, aggregate functions, and UNION ALL.

Query 4: Club Performance by Competition (Level 2)

sql

SELECT

c.name AS club_name,

comp.name AS competition_name,

COUNT(DISTINCT g.game_id) AS matches_played,

SUM(CASE WHEN g.home_club_id = c.club_id AND g.home_score > g.away_score THEN

1

WHEN g.away_club_id = c.club_id AND g.away_score > g.home_score THEN 1

ELSE 0 END) AS wins,

SUM(CASE WHEN g.home_club_id = c.club_id AND g.home_score = g.away_score THEN

1

WHEN g.away_club_id = c.club_id AND g.away_score = g.home_score THEN 1

ELSE 0 END) AS draws,

SUM(CASE WHEN g.home_club_id = c.club_id AND g.home_score < g.away_score THEN

1

WHEN g.away_club_id = c.club_id AND g.away_score < g.home_score THEN 1

ELSE 0 END) AS losses,

SUM(CASE WHEN g.home_club_id = c.club_id THEN g.home_score ELSE g.away_score
END) AS goals_for,

SUM(CASE WHEN g.home_club_id = c.club_id THEN g.away_score ELSE g.home_score
END) AS goals_against,

ROUND(AVG(CASE WHEN g.home_club_id = c.club_id THEN g.home_score ELSE
g.away_score END), 2) AS avg_goals_for

FROM Club c

```
JOIN Game g ON (g.home_club_id = c.club_id OR g.away_club_id = c.club_id)
JOIN Competition comp ON g.competition_id = comp.competition_id
WHERE c.name = 'Manchester City'
GROUP BY c.club_id, c.name, comp.competition_id, comp.name
ORDER BY comp.name;
```

Description: Calculates win-loss-draw records, goals for/against, and average goals for a specific club across each competition they've played in. Team managers and analysts can evaluate club performance across different competitions (league vs. cup vs. international) to identify strengths or weaknesses.

Query Level Justification: Level 2 - Uses multiple joins (Club, Game, Competition), CASE statements for conditional aggregation, aggregate functions (SUM, AVG, COUNT), and GROUP BY.

Query 5: Top Scorers by Season (Level 2)

```
sql
SELECT
    p.name AS player_name,
    c.name AS club_name,
    g.season,
    COUNT(DISTINCT g.game_id) AS games_played,
    SUM(a.goals) AS total_goals,
    ROUND(AVG(a.goals), 2) AS avg_goals_per_game,
    SUM(a.assists) AS total_assists,
    SUM(a.minutes_played) AS total_minutes,
    ROUND(SUM(a.goals) * 90.0 / NULLIF(SUM(a.minutes_played), 0), 2) AS goals_per_90
FROM Player p
JOIN Appearance a ON p.player_id = a.player_id
JOIN Game g ON a.game_id = g.game_id
JOIN Club c ON p.current_club_id = c.club_id
WHERE g.season = 2024
```

```
GROUP BY p.player_id, p.name, c.name, g.season
HAVING SUM(a.goals) >= 10
ORDER BY total_goals DESC
LIMIT 15;
```

Description: Finds the top 15 goal scorers for the 2024 season, showing goals, assists, games played, and goals per 90 minutes. Users can identify the most prolific goal scorers in a given season. The goals-per-90 metric provides better context for players with fewer minutes.

Query Level Justification: Level 2 - Uses multiple joins (Player, Appearance, Game, Club), aggregate functions, HAVING clause, and complex calculation for goals per 90.

Query 6: Transfer Activity Between Clubs (Level 3)

sql

```
SELECT
    p.name AS player_name,
    from_c.name AS from_club,
    to_c.name AS to_club,
    t.transfer_date,
    t.transfer_fee,
    t.season,
    pv_before.market_value_eur AS value_before_transfer,
    pv_after.market_value_eur AS value_after_transfer,
    CASE
        WHEN pv_before.market_value_eur IS NOT NULL AND t.transfer_fee IS NOT NULL
        THEN ROUND(((t.transfer_fee - pv_before.market_value_eur) /
pv_before.market_value_eur) * 100, 2)
        ELSE NULL
    END AS fee_vs_market_value_pct
FROM Transfer t
JOIN Player p ON t.player_id = p.player_id
JOIN Club from_c ON t.from_club_id = from_c.club_id
JOIN Club to_c ON t.to_club_id = to_c.club_id
LEFT JOIN PlayerValuation pv_before ON p.player_id = pv_before.player_id
    AND pv_before.date <= t.transfer_date
LEFT JOIN PlayerValuation pv_after ON p.player_id = pv_after.player_id
    AND pv_after.date > t.transfer_date
    AND pv_after.date = (
        SELECT MIN(date)
        FROM PlayerValuation
        WHERE player_id = p.player_id AND date > t.transfer_date
    )
WHERE t.transfer_fee > 50000000
```



```
ORDER BY t.transfer_fee DESC
LIMIT 20;
```

Description: Shows high-value transfers (over €50M) with the player's market value before and after the transfer, and calculates how the transfer fee compares to market value.

Why it's useful: Scouts and analysts can evaluate whether clubs are overpaying or getting value for players. The market value comparison helps identify transfer market inefficiencies.

Query Level Justification: Level 3 - Uses multiple joins, subquery in the FROM clause (to find valuation after transfer), complex CASE statements, and conditional logic.

Query 7: Player Performance Comparison by Position (Level 2)

SQL Query:

```
sql
SELECT
    p.position,
    COUNT(DISTINCT p.player_id) AS player_count,
    SUM(a.goals) AS total_goals,
    AVG(a.goals) AS avg_goals,
    SUM(a.assists) AS total_assists,
    AVG(a.assists) AS avg_assists,
    AVG(a.minutes_played) AS avg_minutes_played,
    SUM(CASE WHEN a.goals > 0 THEN 1 ELSE 0 END) AS appearances_with_goal,
    ROUND(SUM(CASE WHEN a.goals > 0 THEN 1 ELSE 0 END) * 100.0 /
    COUNT(a.appearance_id), 2) AS goal_scoring_pct
FROM Player p
JOIN Appearance a ON p.player_id = a.player_id
WHERE a.minutes_played > 0
GROUP BY p.position
```

```
HAVING COUNT(DISTINCT p.player_id) >= 50  
ORDER BY avg_goals DESC;
```

Description: Aggregates performance statistics by position, showing which positions contribute most to goals, assists, and overall production.

Why it's useful: Fantasy football users can identify which positions are most productive, helping with team composition decisions.

Query Level Justification: Level 2 - Uses multiple joins, aggregate functions, CASE statements, GROUP BY, and HAVING with aggregate condition.

Query 8: Player's Career Club History (Level 2)

SQL Query:

```
sql  
SELECT  
    p.name AS player_name,  
    t.transfer_date,  
    from_c.name AS from_club,  
    to_c.name AS to_club,  
    t.transfer_fee,  
    t.season,  
    COUNT(DISTINCT a.game_id) AS games_at_club,  
    SUM(a.goals) AS goals_at_club,  
    SUM(a.assists) AS assists_at_club,  
    ROUND(SUM(a.goals) * 90.0 / NULLIF(SUM(a.minutes_played), 0), 2) AS  
goals_per_90_at_club  
FROM Player p  
JOIN Transfer t ON p.player_id = t.player_id  
LEFT JOIN Club from_c ON t.from_club_id = from_c.club_id
```

```

LEFT JOIN Club to_c ON t.to_club_id = to_c.club_id
LEFT JOIN Appearance a ON p.player_id = a.player_id AND a.club_id = t.to_club_id
    AND a.game_id IN (SELECT game_id FROM Game WHERE season = t.season)
WHERE p.name LIKE '%Haaland%'
GROUP BY p.name, t.transfer_id, t.transfer_date, from_c.name, to_c.name, t.transfer_fee,
t.season
ORDER BY t.transfer_date;

```

Description: Shows a player's complete career history with transfers, including performance statistics at each club (games, goals, assists, goals per 90).

Why it's useful: Users can see a player's career trajectory and how they performed at each club, which is valuable for scouts evaluating a player's adaptability.

Query Level Justification: Level 2 - Uses multiple joins, subquery in WHERE clause, aggregate functions, and complex calculations.

—

Query 9: Most Valuable Players by Age Group (Level 3)

SQL Query:

```

sql
SELECT
    age_group.age_range,
    age_group.player_name,
    age_group.club_name,
    age_group.market_value_eur,
    age_group.position,
    age_group.date_of_birth
FROM (
    SELECT
        CASE

```

```

        WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) BETWEEN 18 AND
21 THEN '18-21'
        WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) BETWEEN 22 AND
25 THEN '22-25'
        WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) BETWEEN 26 AND
29 THEN '26-29'
        WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) >= 30 THEN '30+'
        ELSE 'Unknown'
    END AS age_range,
    p.name AS player_name,
    c.name AS club_name,
    pv.market_value_eur,
    p.position,
    p.date_of_birth,
    ROW_NUMBER() OVER (PARTITION BY
        CASE
            WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) BETWEEN 18
AND 21 THEN '18-21'
            WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) BETWEEN 22
AND 25 THEN '22-25'
            WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) BETWEEN 26
AND 29 THEN '26-29'
            WHEN TIMESTAMPDIFF(YEAR, p.date_of_birth, CURDATE()) >= 30 THEN '30+'
            ELSE 'Unknown'
        END
        ORDER BY pv.market_value_eur DESC
    ) AS rank_in_age_group
FROM Player p
JOIN Club c ON p.current_club_id = c.club_id
JOIN PlayerValuation pv ON p.player_id = pv.player_id

```

```

WHERE pv.date = (SELECT MAX(date) FROM PlayerValuation pv2 WHERE pv2.player_id
= p.player_id)
) AS age_group
WHERE rank_in_age_group <= 5
ORDER BY
    FIELD(age_range, '18-21', '22-25', '26-29', '30+'),
    market_value_eur DESC;

```

Description: Finds the top 5 most valuable players in each age group (18-21, 22-25, 26-29, 30+), using the most recent valuation.

Why it's useful: Scouts and team managers can identify the most valuable young talent (for investment) and experienced players (for immediate impact) in different age brackets.

Query Level Justification: Level 3 - Uses subquery in FROM clause, multiple window functions (ROW_NUMBER with PARTITION BY), complex CASE statements for age grouping, and multiple joins.

—

Query 10: Players with Best Goal Contribution Per Minute (Level 2)

SQL Query:

```

sql
SELECT
    p.name AS player_name,
    c.name AS club_name,
    SUM(a.minutes_played) AS total_minutes,
    SUM(a.goals) AS total_goals,
    SUM(a.assists) AS total_assists,
    SUM(a.goals) + SUM(a.assists) AS goal_contributions,
    ROUND((SUM(a.goals) + SUM(a.assists)) * 90.0 / NULLIF(SUM(a.minutes_played), 0), 2)
AS contributions_per_90,

```

```
COUNT(DISTINCT a.game_id) AS games_played,  
ROUND(SUM(a.minutes_played) / 90.0, 2) AS matches_90_equivalent  
FROM Player p  
JOIN Appearance a ON p.player_id = a.player_id  
JOIN Club c ON p.current_club_id = c.club_id  
JOIN Game g ON a.game_id = g.game_id  
WHERE g.season = 2024  
AND SUM(a.minutes_played) >= 900 -- At least 10 full matches  
GROUP BY p.player_id, p.name, c.name  
HAVING SUM(a.goals) + SUM(a.assists) >= 10  
ORDER BY contributions_per_90 DESC  
LIMIT 15;
```

Description: Finds players with the best goal contribution rate (goals + assists per 90 minutes) for the 2024 season, requiring at least 900 minutes played and 10 contributions.

Why it's useful: Identifies the most efficient attacking players who make the biggest impact when they're on the pitch. Valuable for scouts and fantasy managers.

Query Level Justification: Level 2 - Uses multiple joins (Player, Appearance, Club, Game), aggregate functions, HAVING with complex conditions, and advanced calculations.

3. Stored Procedures

Stored Procedure 1: Add Player Performance Record

Code:

sql

DELIMITER //

```
CREATE PROCEDURE AddPlayerPerformance(
    IN p_player_id INT,
    IN p_game_id INT,
    IN p_club_id INT,
    IN p_minutes_played INT,
    IN p_goals INT,
    IN p_assists INT,
    IN p_yellow_cards INT,
    IN p_red_cards INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error adding player performance
record';
    END;

    START TRANSACTION;

    -- Validate player exists
    IF NOT EXISTS (SELECT 1 FROM Player WHERE player_id = p_player_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Player ID does not exist';
```

```

END IF;

-- Validate game exists
IF NOT EXISTS (SELECT 1 FROM Game WHERE game_id = p_game_id) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Game ID does not exist';
END IF;

-- Validate club exists
IF NOT EXISTS (SELECT 1 FROM Club WHERE club_id = p_club_id) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Club ID does not exist';
END IF;

-- Check for duplicate appearance
IF EXISTS (SELECT 1 FROM Appearance WHERE player_id = p_player_id AND game_id
= p_game_id) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'This player already has an
appearance record for this game';
END IF;

-- Insert the appearance record
INSERT INTO Appearance (
    player_id, game_id, club_id, minutes_played,
    goals, assists, yellow_cards, red_cards
) VALUES (
    p_player_id, p_game_id, p_club_id, p_minutes_played,
    p_goals, p_assists, p_yellow_cards, p_red_cards
);

COMMIT;

SELECT 'Performance record added successfully' AS status;

END//

```


DELIMITER ;

Purpose: Adds a new player appearance record with performance statistics, including validation and duplicate checking.

Parameters:

p_player_id: Player's unique identifier

p_game_id: Game's unique identifier

p_club_id: Club the player represented

p_minutes_played: Minutes played in the match

p_goals: Goals scored

p_assists: Assists provided

p_yellow_cards: Yellow cards received

p_red_cards: Red cards received

Stored Procedure 2: Update Player Market Value

Code:

sql

DELIMITER //

```
CREATE PROCEDURE UpdatePlayerValuation(  
    IN p_player_id INT,  
    IN p_valuation_date DATE,  
    IN p_market_value BIGINT,  
    IN p_club_id INT
```

```

)
BEGIN
    DECLARE existing_valuation_count INT;
    DECLARE previous_value BIGINT;
    DECLARE value_change_pct DECIMAL(10,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error updating player valuation';
    END;

    START TRANSACTION;

    -- Validate player exists
    IF NOT EXISTS (SELECT 1 FROM Player WHERE player_id = p_player_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Player ID does not exist';
    END IF;

    -- Check if valuation exists for this date
    SELECT COUNT(*) INTO existing_valuation_count
    FROM PlayerValuation
    WHERE player_id = p_player_id AND date = p_valuation_date;

    IF existing_valuation_count > 0 THEN
        -- Update existing valuation
        SELECT market_value_eur INTO previous_value
        FROM PlayerValuation
        WHERE player_id = p_player_id AND date = p_valuation_date;

        UPDATE PlayerValuation

```

```

SET market_value_eur = p_market_value,
    current_club_id = p_club_id
WHERE player_id = p_player_id AND date = p_valuation_date;

-- Calculate percentage change
IF previous_value IS NOT NULL AND previous_value > 0 THEN
    SET value_change_pct = ((p_market_value - previous_value) / previous_value) * 100;
    SELECT CONCAT('Valuation updated. Change: ', ROUND(value_change_pct, 2), '%')
AS status;
ELSE
    SELECT 'Valuation updated with new value' AS status;
END IF;
ELSE
    -- Insert new valuation
    INSERT INTO PlayerValuation (player_id, date, market_value_eur, current_club_id)
    VALUES (p_player_id, p_valuation_date, p_market_value, p_club_id);
    SELECT 'New valuation record created' AS status;
END IF;

COMMIT;
END//

DELIMITER ;

```

Purpose: Inserts or updates a player's market value on a specific date, with automatic change detection.

Parameters:

p_player_id: Player's unique identifier

p_valuation_date: Date of the valuation

p_market_value: Market value in euros

p_club_id: Current club of the player at valuation time

Stored Procedure 3: Get Player Career Summary

Code:

sql

DELIMITER //

```
CREATE PROCEDURE GetPlayerCareerSummary(
```

```
    IN p_player_id INT,
```

```
    OUT p_player_name VARCHAR(150),
```

```
    OUT p_total_games INT,
```

```
    OUT p_total_goals INT,
```

```
    OUT p_total_assists INT,
```

```
    OUT p_total_minutes INT,
```

```
    OUT p_clubs_count INT,
```

```
    OUT p_current_club VARCHAR(100),
```

```
    OUT p_total_market_value BIGINT
```

```
)
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error retrieving player career  
summary';
```

```
END;
```

```
-- Get basic player info
```

```
SELECT p.name, c.name
```

```
INTO p_player_name, p_current_club
```

```
FROM Player p
```

```
LEFT JOIN Club c ON p.current_club_id = c.club_id
```

```
WHERE p.player_id = p_player_id;
```

```
IF p_player_name IS NULL THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Player not found';
```

```
END IF;
```

```
-- Get career statistics
```

```
SELECT
```

```
    COUNT(DISTINCT a.game_id),
```

```
    SUM(a.goals),
```

```
SUM(a.assists),

SUM(a.minutes_played),

COUNT(DISTINCT a.club_id),

MAX(pv.market_value_eur)

INTO p_total_games, p_total_goals, p_total_assists,

    p_total_minutes, p_clubs_count, p_total_market_value

FROM Appearance a

LEFT JOIN PlayerValuation pv ON a.player_id = pv.player_id

WHERE a.player_id = p_player_id;

END//
```

DELIMITER ;

Purpose: Returns a comprehensive career summary for a player including total games, goals, assists, minutes, clubs played for, current club, and highest market value.

Parameters:

- **p_player_id** (IN): Player's unique identifier
- **p_player_name** (OUT): Player's full name
- **p_total_games** (OUT): Total games played
- **p_total_goals** (OUT): Total goals scored
- **p_total_assists** (OUT): Total assists
- **p_total_minutes** (OUT): Total minutes played
- **p_clubs_count** (OUT): Number of different clubs played for

- `p_current_club` (OUT): Current club name
- `p_total_market_value` (OUT): Highest market value achieved

Stored Procedure 4: Transfer Player Between Clubs

Code:

sql

DELIMITER //

```
CREATE PROCEDURE TransferPlayer(
```

```
    IN p_player_id INT,
```

```
    IN p_from_club_id INT,
```

```
    IN p_to_club_id INT,
```

```
    IN p_transfer_date DATE,
```

```
    IN p_transfer_fee BIGINT,
```

```
    IN p_season VARCHAR(10)
```

```
)
```

```
BEGIN
```

```
    DECLARE current_club INT;
```

```
    DECLARE player_exists INT DEFAULT 0;
```

```
    DECLARE to_club_exists INT DEFAULT 0;
```

```
DECLARE from_club_matches INT DEFAULT 0;
```

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error processing player transfer';
```

```
END;
```

```
START TRANSACTION;
```

```
-- Validate player exists
```

```
SELECT COUNT(*) INTO player_exists FROM Player WHERE player_id = p_player_id;
```

```
IF player_exists = 0 THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Player ID does not exist';
```

```
END IF;
```

```
-- Validate destination club exists
```

```
SELECT COUNT(*) INTO to_club_exists FROM Club WHERE club_id = p_to_club_id;
```

```
IF to_club_exists = 0 THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Destination club does not exist';
```


END IF;

-- Check if from_club_id matches current club

SELECT current_club_id INTO current_club FROM Player WHERE player_id =
p_player_id;

IF p_from_club_id IS NOT NULL AND (current_club IS NULL OR current_club !=
p_from_club_id) THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Player is not currently at the
specified from club';

END IF;

-- Insert transfer record

INSERT INTO Transfer (

player_id, from_club_id, to_club_id,

transfer_date, transfer_fee, season

) VALUES (

p_player_id, current_club, p_to_club_id,

p_transfer_date, p_transfer_fee, p_season

);

-- Update player's current club

```
UPDATE Player SET current_club_id = p_to_club_id WHERE player_id = p_player_id;

COMMIT;

SELECT CONCAT('Player transferred successfully to club ID ', p_to_club_id) AS status;

END//
```

DELIMITER ;

Purpose: Processes a player transfer, recording the transfer details and updating the player's current club. Validates that the player is at the correct source club.

Parameters:

- **p_player_id**: Player's unique identifier
- **p_from_club_id**: Source club (must match current club)
- **p_to_club_id**: Destination club
- **p_transfer_date**: Date of the transfer
- **p_transfer_fee**: Transfer fee in euros
- **p_season**: Season of the transfer

Stored Procedure 5: Get Top Performers by Position

Code:

sql

DELIMITER //

```
CREATE PROCEDURE GetTopPerformersByPosition(  
  
    IN p_position VARCHAR(30),  
  
    IN p_season INT,  
  
    IN p_min_games INT,  
  
    IN p_limit INT  
  
)  
  
BEGIN  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  
    BEGIN  
  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error retrieving top performers';  
  
    END;  
  
  
    IF p_position IS NULL OR p_position = " THEN  
  
        -- All positions  
  
        SELECT  
  
            p.player_id,  
  
            p.name AS player_name,  
  
            c.name AS club_name,  
  
            p.position,
```

```

COUNT(DISTINCT a.game_id) AS games_played,

SUM(a.goals) AS total_goals,

SUM(a.assists) AS total_assists,

SUM(a.goals) + SUM(a.assists) AS goal_contributions,

ROUND((SUM(a.goals) + SUM(a.assists)) * 90.0 / NULLIF(SUM(a.minutes_played),
0), 2) AS contributions_per_90,

ROUND(SUM(a.minutes_played) / 90.0, 2) AS full_match_equivalent

FROM Player p

JOIN Appearance a ON p.player_id = a.player_id

JOIN Club c ON p.current_club_id = c.club_id

JOIN Game g ON a.game_id = g.game_id

WHERE g.season = p_season

GROUP BY p.player_id, p.name, c.name, p.position

HAVING COUNT(DISTINCT a.game_id) >= p_min_games

ORDER BY goal_contributions DESC

LIMIT p_limit;

ELSE

-- Specific position

SELECT

    p.player_id,

```

```
p.name AS player_name,

c.name AS club_name,

p.position,

COUNT(DISTINCT a.game_id) AS games_played,

SUM(a.goals) AS total_goals,

SUM(a.assists) AS total_assists,

SUM(a.goals) + SUM(a.assists) AS goal_contributions,

ROUND((SUM(a.goals) + SUM(a.assists)) * 90.0 / NULLIF(SUM(a.minutes_played),
0), 2) AS contributions_per_90,

ROUND(SUM(a.minutes_played) / 90.0, 2) AS full_match_equivalent

FROM Player p

JOIN Appearance a ON p.player_id = a.player_id

JOIN Club c ON p.current_club_id = c.club_id

JOIN Game g ON a.game_id = g.game_id

WHERE g.season = p_season

AND p.position = p_position

GROUP BY p.player_id, p.name, c.name, p.position

HAVING COUNT(DISTINCT a.game_id) >= p_min_games

ORDER BY goal_contributions DESC

LIMIT p_limit;
```

END IF;

END//

DELIMITER ;

Purpose: Returns the top performers for a given position and season, using goal contributions as the primary metric.

Parameters:

- **p_position:** Position to filter (NULL or empty for all positions)
- **p_season:** Season year to filter
- **p_min_games:** Minimum games played requirement
- **p_limit:** Maximum number of results

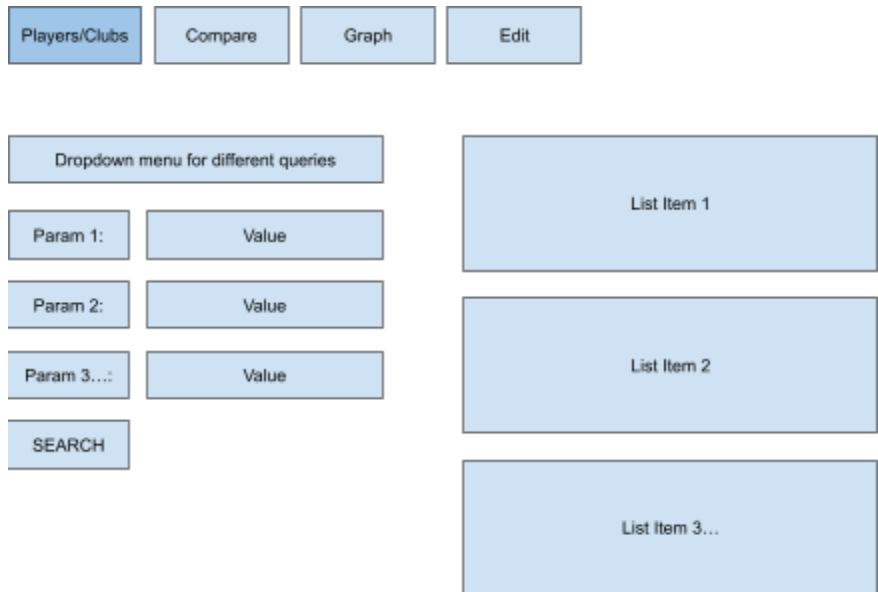
4. Interface Progress

Mock Up Design:

- There will be 4 separate tabs that the user can click on for a different UI with a set purpose
 - Players/Clubs
 - Lists/searches the players/clubs who match the criteria provided (criteria can range from the different list settings for different applications).
 - Queries/Store Procedures that will be in this section
 - **Query 1:** Filters player age and goal average
 - **Query 4:** Searches clubs with specific filters
 - **Query 5:** Filters players by season and sorts by top scorers
 - **Query 9:** Filters players by age group and sorts by top scorers
 - **Query 10:** Filters players by season and sorts by goals per minute

- **Stored Procedure 3:** Views the specified player's career summary
- **Stored Procedure 5:** Filters players by season, position, and sorts by goal contributions

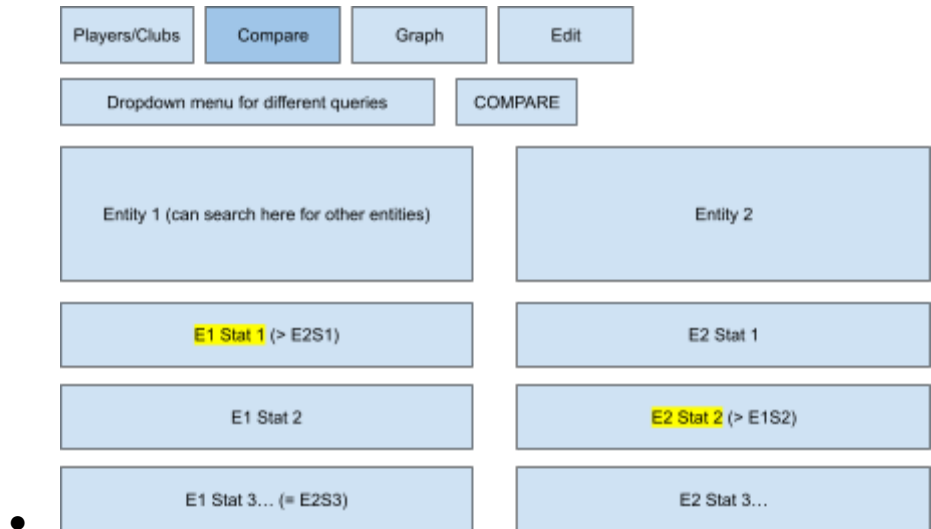
■ UI Diagram



○ Compare

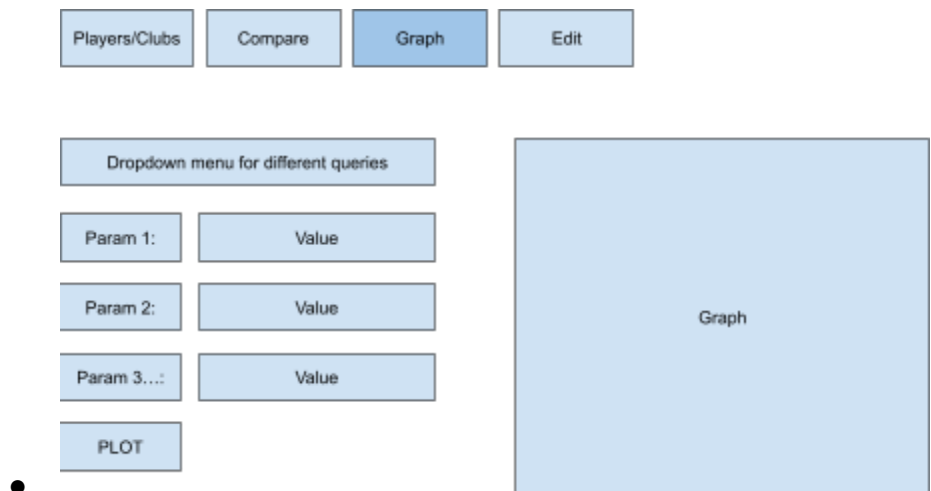
- 2 entities such as players or clubs have their statistics compared side by side with highlighted stats being better than the other entity
- Queries/Store Procedures that will be in this section
 - **Query 3:** Compare players
 - **Query 7:** Compare positions from players
 - **Query 8:** Compare players from before and after a transfer

■ UI Diagram



○ Graph

- Graphically models values over time where a simple list would not suffice.
- Queries/Store Procedures that will be in this section
 - **Query 2:** Player value over time
 - **Query 6:** Player value before and after transfer time
- UI Diagram

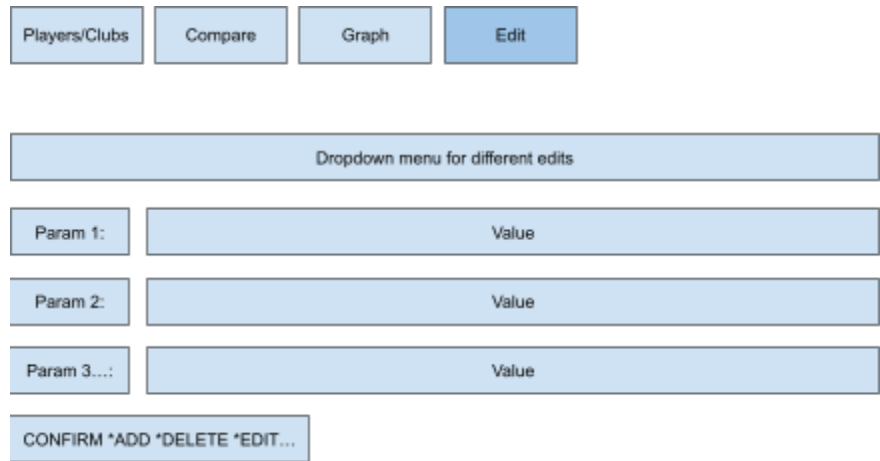


○ Edit

- Used to edit the dataset via add, remove, or other operations.
- Queries/Store Procedures that will be in this section
 - **Stored Procedure 1:** Adds a player
 - **Stored Procedure 2:** Edits a player's market value

- **Stored Procedure 4:** Adds the transfer data for a player between 2 clubs

■ UI Diagram



- Current implementation progress
 - Have basic tab setup leading to the different pages for the queries.
 - App.js

```

import { useState } from 'react';
import './App.css';

function App() {
  const tabs = [
    {name:"Players/Clubs"},
    {name:"Compare"},
    {name:"Graph"},
    {name:"Edit"}
  ];

  const [activeTab, setActiveTab] = useState(0);

  return (
    <div className="App">
      <header className="App-header">

```

```

    <div className="tab-menu">
      {tabs.map((tab, index) => (
        <div
          key={tab.name}
          className={`tab-box ${activeTab === index ?
"selected" : ""}`}
          onClick={() => setActiveTab(index)}
        >
          {tab.name}
        </div>
      ))}
    </div>

    <div className="tab-section">
      <h2>{tabs[activeTab].name} Section</h2>
    </div>

  </header>
</div>
);
}

export default App;

```

○ App.css

```

html, body {
  margin: 0;
  padding: 0;
  font-family: 'Trebuchet MS', sans-serif;
}

.App {
  text-align: center;

```

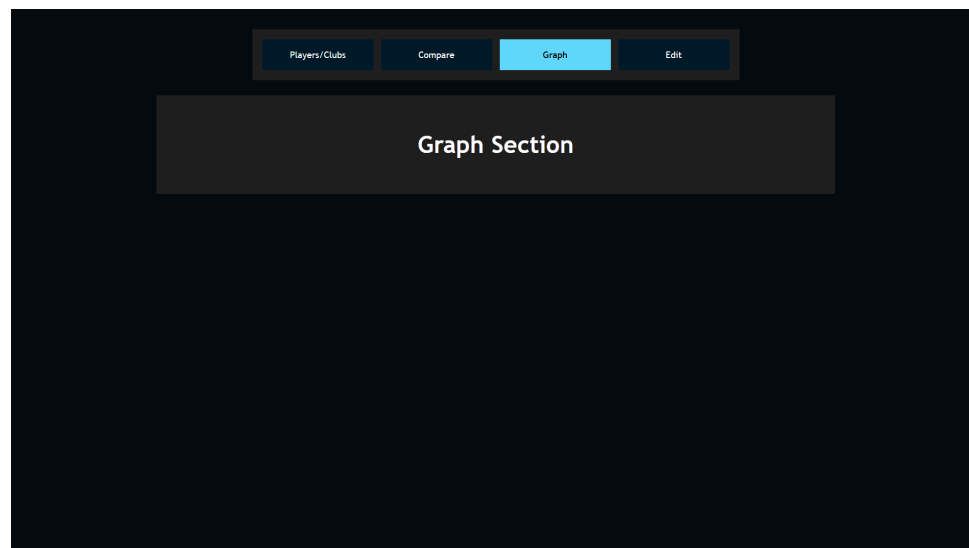
```
○ }
○
○
○ .App-header {
○   background-color: #090c12;
○   min-height: 100vh;
○   display: flex;
○   flex-direction: column;
○   align-items: center;
○   padding-top: 40px;
○   font-size: calc(10px + 2vmin);
○   color: white;
○ }
○
○
○ .tab-menu {
○   display: flex;
○   justify-content: center;
○   gap: 15px;
○   margin-bottom: 30px;
○ }
○
○
○ .tab-box {
○   flex: 1;
○   width: 180px;
○   padding: 20px;
○   background-color: #001b2a;
○   color: white;
○   cursor: pointer;
○   font-size: 18px;
○   font-weight: 500;
○ }
○
○
○ .tab-box:hover {
○   background-color: #73899c;
```

```

○ }
○
○ .tab-box.selected {
○   background-color: #61dafb;
○   color: #000;
○ }
○
○ .tab-section {
○   width: 70%;
○   min-height: 150px;
○   padding: 30px;
○   background-color: #222;
○   box-sizing: border-box;
○ }

```

○ Visual:



○

5. Testing Plan

The main objectives of testing are to verify that good inputs produce the expected outputs and that the invalid inputs produce an error message.

Test Case	Feature Tested	Input	Expected Result	Pass/Fail Criteria
-----------	----------------	-------	-----------------	--------------------

1	Top scorers from season	2022 season	Sorted list for the top scorers from the 2022 season	Results match historical data
2	Insert new player appearance	Valid player id scoring 1000 goals in a single game	Success message stating that the entry has been added	Top scorer stats now shows the valid player as the top scorer
3	Update player market value	Valid player id valued at 1 billion euros	Success message stating that the market value has been updated	Market value history has a 1 billion euro entry
4	Invalid career history search	Search invalid player id's club career history	Search is invalid	DBMS reports error for the id
5	Insert invalid player appearance	Insert a player appearance with an invalid player id	Insert is invalid	DBMS reports error for the id

6. Baseline Query Performance for Checkpoint 4

7. Updates to Project Design or Management

No major changes to the project design or management plan at this time

8. AI-Use Statement

- Used AI to help research, debug, and create the basic UI setup
 - Accepted suggestions that resulted in further development of the UI such as having the correct width of the gray background for the tab sections.
 - Verified work by using existing knowledge about the material